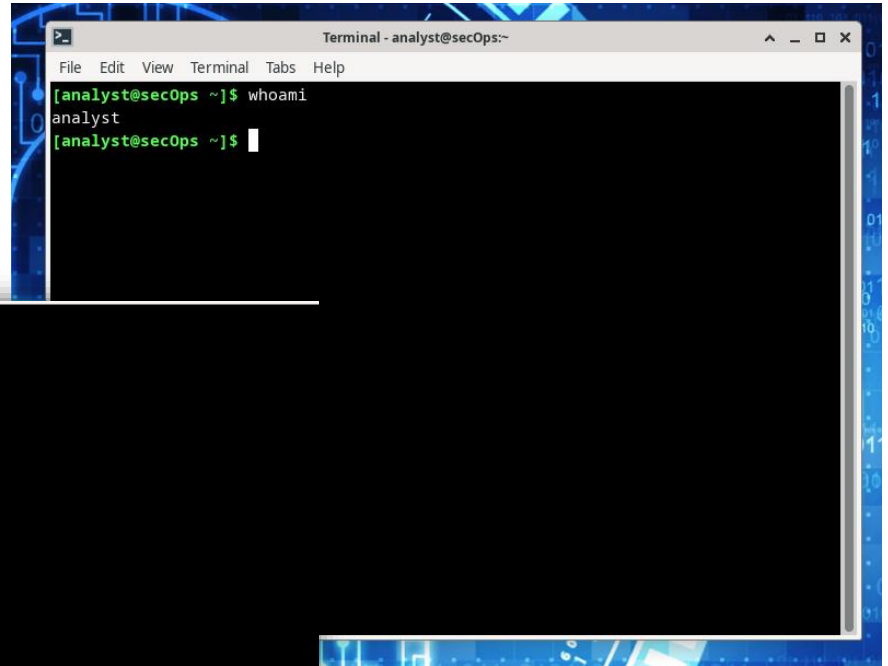


CYBEROPS VM MININET WIRESHARK PCAP TRAFFIC ANALYSIS:

Analisi del traffico tra due host (H1 Client, H4 server http nginx) di mininet su macchina CyberOps tramite Wireshark.

Accesso come user:

analyst e avvio di mininet con topologia pre-stabilita



CyberOPS Topology:

```

      -----
      | R1 |-----| H4 |
      -----
      |
      |
      -----
      | S1 |-----|
      -----
      |
      |
      -----
      | H1 | | H2 | | H3 |
      -----
  
```

*** Adding internal links
 *** Creating network
 *** Adding hosts:
 H1 H2 H3 H4 R1
 *** Adding switches:
 s1
 *** Adding links:
 (H1, s1) (H2, s1) (H3, s1) (H4, R1) (s1, R1)
 *** Configuring hosts
 H1 H2 H3 H4 R1 Enabling IP forwarding on R1
 *** Starting controller
 *** Starting 1 switches
 s1 ...
 *** Routing Table on Router:
 Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
10.0.0.0	0.0.0.0	255.255.255.0	U	0	0	0	R1-eth1
172.16.0.0	0.0.0.0	255.240.0.0	U	0	0	0	R1-eth2

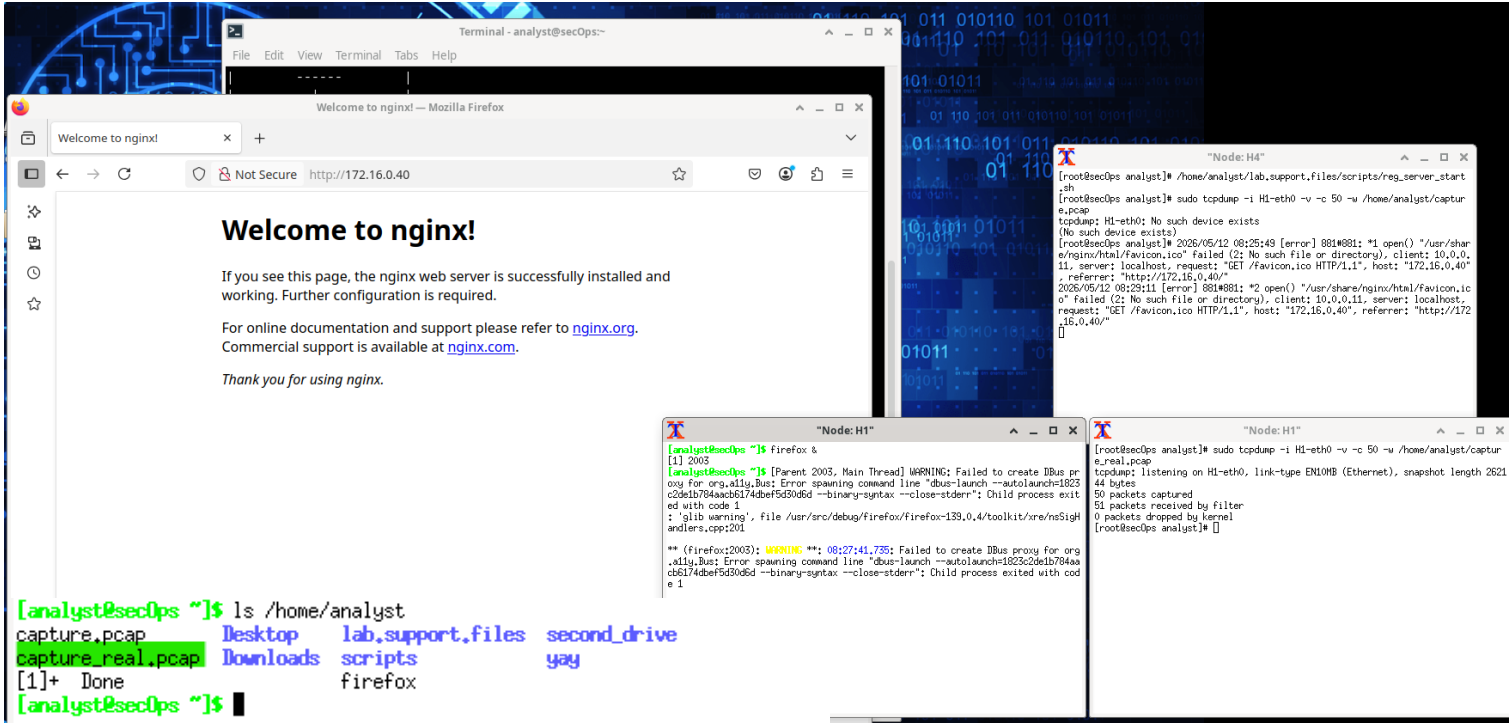
*** Starting CLI:
 mininet>

Con comando CLI "xterm" io "Spawno" i terminali di H1 che ci farà da client e H4 che farà da server http nginx.

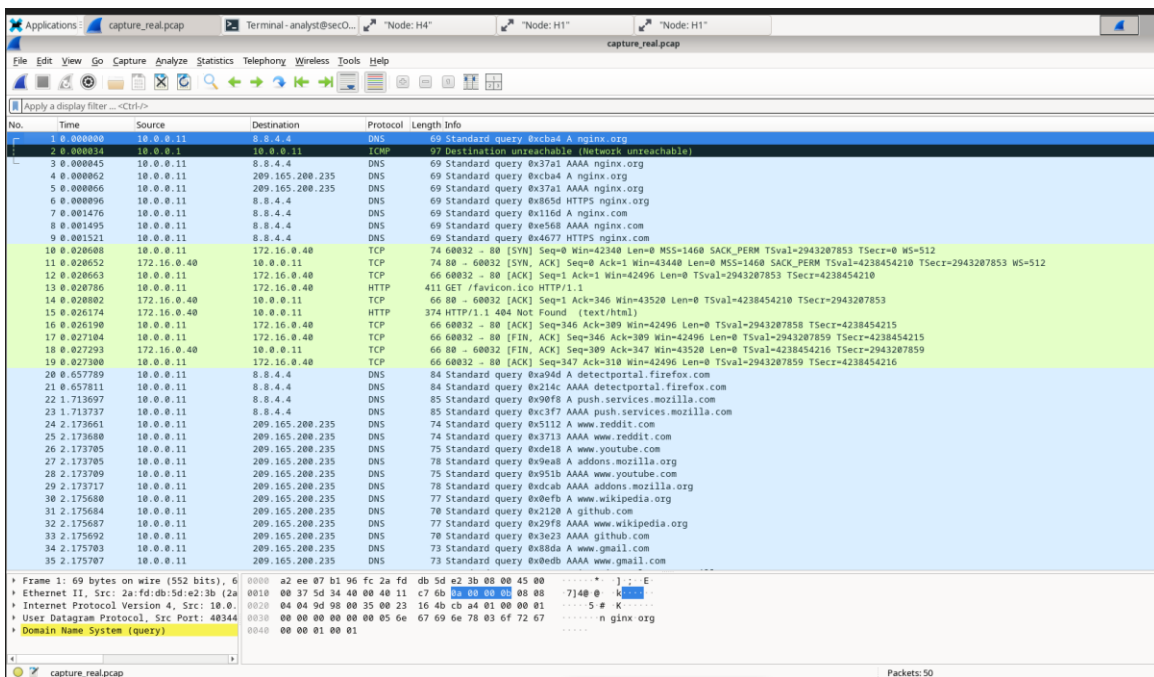
H1 ipv4: 10.0.0.11/24

H4 ipv4: 172.16.0.40/24

Da terminale H4 avvio servizio server con /home/analyst/lab.support.files/reg_server_start.sh the shell script to start the nginx service on the port 80, I'll now go on H1 and get super user to run a firefox instance and visit the ip of H4 and immediately capture the traffic as a pcap with the tool tcpdump as capture_real.pcap



Avvio ora wireshark e metto come file da voler esaminare la cattura appena svolta del traffico su browser firefox, ma soprattutto interessandomi ai frame protocollo TCP per analizzare il three way handshake.



Applico filtro “tcp”, altri filtri utili per analisi sarebbero quelli che abbiamo ad esempio utilizzato in esercizi passati per query di risorse da parte di un IP al server http in questo caso come “http.request”, specificando poi metodi che mettono il servizio a rischio come “DELETE” con http.request.method == “DELETE” ad esempio. Oppure controllare con http.response.code == 401,403 (unauthorized,forbidden) se qualcuno sta cercando di accedere a pagine su cui non ha autorizzazione (probabile probing o enumerazione directories)

No.	Time	Source	Destination	Protocol	Length	Info
10	0.020608	10.0.0.11	172.16.0.40	TCP	74	60032 → 80 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=2943207853 TSecr=0 WS=512
11	0.020652	172.16.0.40	10.0.0.11	TCP	74	80 → 60032 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=4238454210 TSecr=2943207853 WS=512
12	0.020663	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=2943207853 TSecr=4238454210
13	0.020786	10.0.0.11	172.16.0.40	HTTP	411	GET /favicon.ico HTTP/1.1
14	0.020802	172.16.0.40	10.0.0.11	TCP	66	80 → 60032 [ACK] Seq=1 Ack=346 Win=43520 Len=0 TSval=4238454210 TSecr=2943207853
15	0.026174	172.16.0.40	10.0.0.11	HTTP	374	HTTP/1.1 404 Not Found (text/html)
16	0.026190	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=346 Ack=309 Win=42496 Len=0 TSval=2943207858 TSecr=4238454215
17	0.027104	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [FIN, ACK] Seq=346 Ack=309 Win=42496 Len=0 TSval=2943207859 TSecr=4238454215
18	0.027293	172.16.0.40	10.0.0.11	TCP	66	80 → 60032 [FIN, ACK] Seq=309 Ack=347 Win=43520 Len=0 TSval=4238454216 TSecr=2943207859
19	0.027300	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=347 Ack=310 Win=42496 Len=0 TSval=2943207859 TSecr=4238454216

Identifico subito dall'ordine di stream il 3 way handshake e persino il finalize.

No.	Time	Source	Destination	Protocol	Length	Info
10	0.020608	10.0.0.11	172.16.0.40	TCP	74	60032 → 80 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=2943207853 TSecr=0 WS=512
11	0.020652	172.16.0.40	10.0.0.11	TCP	74	80 → 60032 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=4238454210 TSecr=2943207853 WS=512
12	0.020663	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=2943207853 TSecr=4238454210
13	0.020786	10.0.0.11	172.16.0.40	HTTP	411	GET /favicon.ico HTTP/1.1
14	0.020802	172.16.0.40	10.0.0.11	TCP	66	80 → 60032 [ACK] Seq=1 Ack=346 Win=43520 Len=0 TSval=4238454210 TSecr=2943207853
15	0.026174	172.16.0.40	10.0.0.11	HTTP	374	HTTP/1.1 404 Not Found (text/html)
16	0.026190	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=346 Ack=309 Win=42496 Len=0 TSval=2943207858 TSecr=4238454215
17	0.027104	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [FIN, ACK] Seq=346 Ack=309 Win=42496 Len=0 TSval=2943207859 TSecr=4238454215
18	0.027293	172.16.0.40	10.0.0.11	TCP	66	80 → 60032 [FIN, ACK] Seq=309 Ack=347 Win=43520 Len=0 TSval=4238454216 TSecr=2943207859
19	0.027300	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=347 Ack=310 Win=42496 Len=0 TSval=2943207859 TSecr=4238454216

3-way handshake

Finalize + last ACK

Noto quindi che i frame 10-11-12 seguono il pattern di 3 way handshake ovvero:

SYN → SYN-ACK → ACK e che l'inizializzazione della connessione con SYN sia appunto

partita dall'ip 10.0.0.11 di H1 come ci aspettavamo e fosse indirizzata a 172.16.0.40 ovvero il

server H4 in direzione porta 80. Ora vediamo in dettaglio.

10	0.020608	10.0.0.11	172.16.0.40	TCP	74	60032 → 80 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=2943207853 TSecr=0 WS=512
11	0.020652	172.16.0.40	10.0.0.11	TCP	74	80 → 60032 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=4238454210 TSecr=2943207853 WS=512
12	0.020663	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=2943207853 TSecr=4238454210
13	0.020786	10.0.0.11	172.16.0.40	HTTP	411	GET /favicon.ico HTTP/1.1
14	0.020802	172.16.0.40	10.0.0.11	TCP	66	80 → 60032 [ACK] Seq=1 Ack=346 Win=43520 Len=0 TSval=4238454210 TSecr=2943207853
15	0.026174	172.16.0.40	10.0.0.11	HTTP	374	HTTP/1.1 404 Not Found (text/html)
16	0.026190	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=346 Ack=309 Win=42496 Len=0 TSval=2943207858 TSecr=4238454215
17	0.027104	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [FIN, ACK] Seq=346 Ack=309 Win=42496 Len=0 TSval=2943207859 TSecr=4238454215
18	0.027293	172.16.0.40	10.0.0.11	TCP	66	80 → 60032 [FIN, ACK] Seq=309 Ack=347 Win=43520 Len=0 TSval=4238454216 TSecr=2943207859
19	0.027300	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=347 Ack=310 Win=42496 Len=0 TSval=2943207859 TSecr=4238454216

<pre> Frame 10: 74 bytes on wire (592 bits), Ethernet II, Src: 2a:fd:db:5d:e2:3b (2a:fd:db:5d:e2:3b), Dst: 172.16.0.40, Internet Protocol Version 4, Src: 10.0.0.11, Dst: 172.16.0.40, Transmission Control Protocol, Src Port: 60032, Dst Port: 80 Source Port: 60032 Destination Port: 80 [Stream index: 0] [Stream Packet Number: 1] [Conversation completeness: Complete] [TCP Segment Len: 0] Sequence Number: 0 (relative sequence number) Sequence Number (raw): 1076321896 [Next Sequence Number: 1 (relative sequence number)] Acknowledgment Number: 0 Acknowledgment number (raw): 0 1010 ... = Header Length: 40 bytes Flags: 0x002 [SYN] Window: 42340 [Calculated window size: 42340] Checksum: 0xb671 (unverified) [Checksum Status: Unverified] Urgent Pointer: 0 Options: (20 bytes), Maximum segment size, Timestamps </pre>	<pre> 0000 a2 ee 07 b1 96 fc 2a fd db 5d e2 3b 08 00 45 00*...E... 0010 00 3c 7c 16 40 00 40 06 08 63 04 00 00 0b ac 10 < @@... 0020 00 28 ea 80 00 50 40 27 5e 68 00 00 00 0a 02 (...P@Ah... 0030 a5 64 b6 71 00 00 02 04 05 b4 04 02 08 0a af 6d d...m 0040 c9 ad 00 00 00 00 01 03 03 09 </pre>	<pre> Flags: 0x002 (SYN) 000. = Reserved: Not set ...0 = Accurate ECN: Not set ... 0... = Congestion Window Reduced: Not set0.. = ECN-Echo: Not set0. = Urgent: Not set0 = Acknowledgment: Not set0.. = Push: Not set0.. = Reset: Not set1. = Syn: Set0 = Fin: Not set [TCP Flags:S.] Window: 42340 </pre>
--	---	--

Analizzando il primo frame (frame 10) noto prima di tutto che si tratta di un pacchetto TCP ethernet II con indirizzo IPv4 sorgente 10.0.0.11 e diretto a ricevente IPv4 172.16.0.40, nelle informazioni del protocollo TCP vedo che la porta sorgente è una porta effimera 60032 (connessione effimera/temporanea) e che è diretta a porta di destinazione 80, default di servizio http come ci aspettavamo appunto dal server nginx H4; si vede che questo frame sia il primo (seq number 0) RELATIVO a questo stream di frame ma che esso abbia generato il sequence number randomico “1076321896” che tornerà utile per controlli successivi.

Notiamo che l'unico Flag settato sia quello SYN, confermando che si tratta quindi del primo step di inizializzazione di una connessione tramite il 3 way handshake. Possibile inoltre vedere la window size di 42340 bytes disponibili sul receive buffer.

Controlliamo ora il SYN-ACK e l'ACK finale.

Vediamo subito da frame 11 che il source e destination IPv4s si sono invertiti, e nello stesso modo anche le porte di sorgente e destinazione. Vediamo che in questo stream TCP si tratta del SECONDO pacchetto quindi seguente a quello di SYN appena analizzato, ma che utilizzi un numero di sequenza proprio (come standard del 3 way handshake, quello precedente verrà ripreso e incrementato dall'ACK di finalizzazione handshake). Vedo che sono attivi 2 flag ovvero il flag SYN ed il flag ACK. Sembra che a questo punto della connessione il buffer abbia liberato spazio in quanto è aumentato a 43440bytes disponibili.

11	0.020652	172.16.0.40	10.0.0.11	TCP
12	0.020663	10.0.0.11	172.16.0.40	TCP
13	0.020786	10.0.0.11	172.16.0.40	HTTP
14	0.020802	172.16.0.40	10.0.0.11	TCP
15	0.026174	172.16.0.40	10.0.0.11	HTTP
16	0.026190	10.0.0.11	172.16.0.40	TCP
17	0.027104	10.0.0.11	172.16.0.40	TCP
18	0.027293	172.16.0.40	10.0.0.11	TCP
19	0.027300	10.0.0.11	172.16.0.40	TCP

Frame 11: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0

Ethernet II, Src: a2:ee:07:b1:96:fc (a2:ee:07:b1:96:fc), Dst: 02:14:00:00:00:00 (02:14:00:00:00:00)

Internet Protocol Version 4, Src: 172.16.0.40, Dst: 10.0.0.11

Transmission Control Protocol, Src Port: 80, Dst Port: 60032

Source Port: 80

Destination Port: 60032

[Stream index: 0]

[Stream Packet Number: 2]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 0]

Sequence Number: 0 (relative sequence number)

Sequence Number (raw): 294062078

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 1076321897

1010 = Header Length: 40 bytes (10)

Flags: 0x012 (SYN, ACK)

Window: 43440

[Calculated window size: 43440]

Checksum: 0xb671 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

No.	Time	Source	Destination	Protocol	Length	Info
10	0.020608	10.0.0.11	172.16.0.40	TCP	74	60032 → 80 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=2943207853 TSecr=0 WS=512
11	0.020652	172.16.0.40	10.0.0.11	TCP	74	80 → 60032 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=4238454210 TSecr=2943207853
12	0.020786	10.0.0.11	172.16.0.40	HTTP	411	GET /favicon.ico HTTP/1.1
13	0.020802	172.16.0.40	10.0.0.11	TCP	66	80 → 60032 [ACK] Seq=1 Ack=346 Win=43520 Len=0 TSval=4238454210 TSecr=2943207853
14	0.026174	172.16.0.40	10.0.0.11	HTTP	374	HTTP/1.1 404 Not Found (text/html)
15	0.026190	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=346 Ack=309 Win=42496 Len=0 TSval=2943207858 TSecr=4238454215
16	0.027104	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [FIN, ACK] Seq=346 Ack=309 Win=42496 Len=0 TSval=2943207859 TSecr=4238454215
17	0.027293	172.16.0.40	10.0.0.11	TCP	66	80 → 60032 [FIN, ACK] Seq=309 Ack=347 Win=43520 Len=0 TSval=4238454216 TSecr=2943207859
18	0.027300	10.0.0.11	172.16.0.40	TCP	66	60032 → 80 [ACK] Seq=347 Ack=310 Win=42496 Len=0 TSval=2943207859 TSecr=4238454216

Frame 12: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0

Ethernet II, Src: 2a:fd:db:5d:e2:3b (2a:fd:db:5d:e2:3b), Dst: 02:14:00:00:00:00 (02:14:00:00:00:00)

Internet Protocol Version 4, Src: 10.0.0.11, Dst: 172.16.0.40

Transmission Control Protocol, Src Port: 60032, Dst Port: 80

Source Port: 60032

Destination Port: 80

[Stream index: 0]

[Stream Packet Number: 3]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 0]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 1076321897

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 294062079

1000 = Header Length: 32 bytes (8)

Flags: 0x010 (ACK)

Window: 83

[Calculated window size: 42496]

[Window size scaling factor: 512]

Checksum: 0xb669 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

Options: (12 bytes), No-Operation (NOP), No-Operation (NOP)

[Timestamps]

Con l'ultimo frame 12 di ACK notiamo appunto il seq number incrementato di 1 da quello di SYN iniziale, il singolo flag ACK, window size di 42496, source port efimero di H1 verso destination port 80 http di H4. Tutto a norma, la connessione è stata stabilita. Possiamo anche vedere la richiesta di risorse da parte di H1 subito dopo.